

Bundle File Tool v2.1 Build 100

Addendum B — Gap Review, Repository-Awareness Assessment, and File Inclusion/Exclusion Design

Prepared by John, Lead Developer — Proposed for Ringo, George, and Paul — 2026-07-22

Status: Design-only round. No code changes are included or implied by this document. All findings below were verified directly against the current Build 101 source (`config.py`, `validators.py`, `cli.py`, `unbundle_frame.py`, `main_window.py`, `mode_manager.py`, `base.py`, `models.py`, `writer.py`, `plain_marker.py`) before being asserted, per the team's zero-assumption directive.

1. Executive Summary

This addendum responds to three items raised by Ringo against Paul's Build 100 Tri-UI Feature and System Specification (2026-07-08): (1) confirmation that the core/UI separation and CLI automation direction is the right call, (2) whether repository (git) interaction should be added, and (3) a request for granular file inclusion/exclusion by file type, available from the CLI, a config file, and both GUI environments.

Findings, in brief:

- The service-facade and CLI-first direction in Paul's spec is sound and should be endorsed without change — it is also the prerequisite for the file-type filtering feature requested below, since a single filtering implementation must serve CLI, Tkinter, and Web identically.
- Repository interaction has clear value but should not enter Build 100 scope. It is recommended as a Build 101 candidate, read-only and opt-in, to avoid destabilizing the facade/profile/UI-parity work that is already the mandatory scope of Build 100.
- PDF files are not blocked by any special-case rule in the code. There is no PDF-specific logic anywhere in the current source (confirmed by direct search). The behavior Ringo is noticing is a side effect of an allow-list filtering posture, and the Build 100 spec's own proposed default allow-list has the same gap. Section 3 documents this precisely.
- The CLI already supports repeatable `--include/--exclude` glob flags that merge with config-file `safety.allow_globs/deny_globs` (`cli.py`). This is a real, working foundation — the requested capability is an extension of existing behavior, not new territory.
- No file-type discovery or checklist UI exists today in the Tkinter frames reviewed. This is genuinely new work for Build 100/101, not a duplicate of anything already built.

2. Core/UI Separation and CLI Automation — Endorsement

Ringo's instinct here is correct and is already the central architectural move in Paul's spec: a single `BundleToolService` consumed identically by CLI, Tkinter, and Web. Two points are worth stating explicitly for the record:

- Automation value: once `cli.py` is a thin adapter over `BundleToolService` with no direct parser/writer/profile construction (spec §8.2 acceptance rule), the tool becomes safely scriptable — usable from scheduled tasks, pre-commit hooks, or other project tooling (including `pyprojectmgr`'s own automation, kept as a separate project per team convention) without dragging in Tkinter or a browser.
- This same facade is what makes the file-type filtering feature in Section 5 cheap to deliver across all three UIs: the filtering logic is implemented once, in core, and every UI becomes a thin presentation of the same discover/filter operations.
- Recommendation: keep Phase 1 (service facade) as the highest-priority phase, exactly as Paul recommended to John in §14. Do not let the file-type filtering or repository-awareness requests below pull that priority forward or sideways — they should be implemented as facade methods, once the facade exists.

3. Verified Findings on File Exclusion (PDF and Others)

Before proposing a fix, the actual current behavior was confirmed rather than assumed, per team directive.

3.1 No extension-specific or PDF-specific rule exists

A full-text search for "pdf" across every .py file in the current source (models.py, parser.py, writer.py, validators.py, config.py, cli.py, base.py, plain_marker.py, markdown_fence.py, main_window.py, mode_manager.py, unbundle_frame.py) returned zero matches. Binary-vs-text detection in writer.py is content-sniffed, not extension-based: it reads a chunk, checks for null bytes, and falls back to a UTF-8 decode attempt (writer.py, BundleCreator file-reading path). A PDF is simply treated as a binary file and, when treat_binary_as_base64 is true (the default), it is base64-encoded into the bundle like any other binary — it is not rejected by any binary-handling code.

3.2 The real cause: allow-list posture, in two places that already disagree with each other

Two different default allow-list definitions exist in the codebase and the proposed spec, and neither includes .pdf:

Source	Default posture	Effect on PDFs
config.py DEFAULT_CONFIG (current, in source)	safety.allow_globs = ["src/**", "docs/**"] (directory allow-list)	A PDF is included only if it happens to sit under src/ or docs/; otherwise silently dropped.
Build 100 spec §10, Appendix (proposed)	safety.allow_globs = ["**/*.py", "**/*.md", "**/*.json", "**/*.toml", "**/*.html", "**/*.css", "**/*.js"] (extension allow-list)	.pdf is absent from the list, along with .txt, .yaml/.yml, .xml, .csv, .sh, .bat, .ini/.cfg, .sql, .rst, and image types — all silently dropped by default.

Note: These two defaults are also inconsistent with each other in kind (directory-based vs. extension-based), which is itself a migration risk worth flagging to George: a v1.1.5 or current-Build-101 config that migrates forward under the Build 100 schema will change which files get included, with no explicit migration rule covering it in §4.2 (“Version drift”) of the spec.

3.3 Existing CLI foundation (already works today)

cli.py already defines repeatable --include and --exclude glob flags, merged with config-file safety.allow_globs/deny_globs when the flag is not supplied (cli.py, “Merge include/exclude globs” block). GlobFilter (validators.py) implements deny-first precedence with '**' recursive matching via PurePosixPath, and already accepts both the modern allow_patterns/deny_patterns and the legacy allow_globs/deny_globs argument names. This is a solid, tested foundation — the request below extends it rather than replacing it.

3.4 What does not yet exist

- No extension-only convenience flags (e.g., --include-ext pdf,md) — today's users must know full glob syntax.
- No config-file surface for extension-based rules distinct from full glob patterns.
- No GUI or Web UI element of any kind for discovering or toggling file types. unbundle_frame.py's only “filetypes” reference is an OS file-open dialog filter, unrelated to bundle content filtering. main_window.py and mode_manager.py contain no filtering UI. This confirms the requested capability does not already exist under another name — it is safe to propose as new work without risk of duplicating or colliding with an existing function.

Note: bundle_frame.py is referenced throughout Paul's spec and in the team's own change history (CP-2026-002) but was not present in the current project file set reviewed for this addendum. Before Phase 4 work begins, John should re-pull bundle_frame.py from the live tree and re-verify it against the assumptions in this document — do not assume its current state from memory alone.

4. Gap Review of the Build 100 Spec

Items George and Paul should weigh in on before Build 100 implementation begins:

#	Gap	Why it matters	Suggested owner
G1	Config default posture (directory allow-list vs. extension allow-list) is undefined and the two existing defaults disagree.	Silent, unreviewed changes to what gets bundled violate zero-regression discipline and the “formal change management” rule for lifecycle-affecting behavior.	George (architecture decision) + Paul (migration test)

#	Gap	Why it matters	Suggested owner
G2	Service result error taxonomy (§7.4) does not distinguish “file excluded by filter” from “file blocked by safety policy.”	GUI/Web need to show these differently — one is a user choice, the other is a safety guardrail. Conflating them in `warnings`/`errors` will confuse the file-type checklist UI in Section 5.	John, in service.py design
G3	No relationship-table entries planned for new API-function/UI-element pairs the file-type filter will introduce.	Team's Relationship-Based QA/QC directive requires every API function have a documented UI element relationship before QC passes.	Paul (assets.db)
G4	Acceptance matrix (§13) has no test for extension-based include/exclude or for config/CLI/GUI precedence when they conflict.	Without an explicit precedence rule and test, three UIs could silently disagree on which files a given source tree bundles.	Paul
G5	Repository-awareness is absent from §5.4 non-goals list.	Should be added explicitly now (Section 6) so it is not accidentally assumed in-scope by anyone reading only the original spec.	Ringo (approval)

5. File Inclusion/Exclusion by Type — Proposed Design

5.1 Design principle

Reuse the existing, already-tested GlobFilter deny-first engine (validators.py) as the single source of truth. Every new surface — config keys, CLI flags, GUI checklist, Web checklist — translates user intent into allow/deny glob patterns and calls the same filter. No parallel filtering implementation should be written for any UI.

5.2 Resolution order (proposed, needs Ringo/George sign-off)

- 1. Safety-critical deny patterns (non-negotiable): .venv, __pycache__, .git, .pytest_cache, dist, build, and existing bundle-artifact patterns. Always denied regardless of any other setting.
- 2. Config file safety.allow_globs / safety.deny_globs (and new safety.allow_extensions / safety.deny_extensions, see 5.3).
- 3. CLI --include / --exclude (existing) and new --include-ext / --exclude-ext, when supplied, override the config-file lists for that invocation only — config on disk is not modified.
- 4. GUI/Web session-level file-type checklist (new) — a per-operation refinement layered on top of steps 1–3, scoped to the current bundle action only unless the user explicitly saves it back to config via the Settings dialog.

Note: This mirrors the deny-first precedence already implemented in GlobFilter, just extended with one more layer. Formal sign-off on this order is requested from George before Phase 2/4/5 implementation, consistent with the team's change-management rule.

5.3 Config file schema addition (design only — no code in this round)

Proposed additive keys under the existing safety section, translated internally to glob patterns so GlobFilter needs no changes:

```
"safety": {
  "allow_globs": [...],           // existing
  "deny_globs": [...],           // existing
  "allow_extensions": [".py", ".md", ".pdf", ".txt", "..."], // NEW, optional
  "deny_extensions": [".zip", ".exe", "..."],                // NEW, optional
  "max_file_mb": 10             // existing
}
```

If allow_extensions is present, it is expanded to "**/*{ext}" patterns and unioned with allow_globs before the existing GlobFilter runs; same pattern for deny_extensions. This keeps GlobFilter as the single enforcement point and avoids a second filtering code path.

5.4 CLI addition (design only)

```
python src\cli.py bundle <source_paths...> --output <file> [--include-ext .py,.md,.pdf]
[--exclude-ext .zip,.exe] [--include <glob>] [--exclude <glob>] ...
```

--include-ext / --exclude-ext are pure convenience wrappers that expand to the same allow_globs/deny_globs merge already implemented in cli.py — no new filtering logic, only argument translation.

5.5 New service method (once the Build 100 facade exists)

discover_file_types(request) → scans the requested source path(s) (respecting safety-critical denies only, not yet the user's allow/deny choices) and returns the unique set of file extensions found, each with a file count and total size. This single method backs both the Tkinter and Web checklists, keeping them in lockstep by construction.

5.6 Tkinter GUI addition

A “File Types” panel/dialog reachable from the Bundle frame: after Select Source, the user can open a checklist populated from discover_file_types() (extension, count, total size), check/uncheck to include or exclude, and the current preview/create-bundle actions apply the resulting filter through the same BundleToolService call already used for those actions. No direct calls to core.validators from ui/bundle_frame.py, consistent with the spec's §6.1 dependency rule.

5.7 Web UI addition

New endpoint and DOM IDs, added to the lists already defined in Paul's spec §8.4:

Element	Type	Purpose
/api/files/types	POST endpoint	Wraps discover_file_types(); returns extension/count/size list
panelFileTypes	DOM ID	Container for the checklist panel
fileTypeChecklist	DOM ID	Checklist of discovered extensions
btnApplyFileTypes	DOM ID	Applies checklist selection to the active bundle preview/create request

5.8 Default extension allow-list correction

Recommend closing gap G1 by adopting a deny-list default posture rather than an allow-list default: deny only safety-critical noise (build artifacts, VCS/cache directories, previous bundle outputs, archives) and permit everything else — including .pdf, .txt, .yaml/.yml, .xml, .csv, .rst, .ini/.cfg, .sql, .sh, .bat — unless explicitly excluded. This matches how most users will actually reason about the tool (“bundle everything except the junk”) and removes the silent-drop behavior that prompted Ringo's question. This is a default-behavior decision, not a mechanism change, and is flagged here for Ringo/George approval rather than assumed.

5.9 New acceptance tests (additive to Paul's §13 matrix)

Test ID	Category	Scenario	Expected result
BFT-B100-016	Filtering	Config safety.allow_extensions includes .pdf; bundle a tree containing a PDF.	PDF is included as a binary/base64 entry.
BFT-B100-017	Filtering	CLI --exclude-ext .pdf overrides config allow_extensions for one invocation.	PDF excluded; config file on disk unchanged.
BFT-B100-018	Filtering	GUI/Web discover_file_types() on canonical sample project.	Returned extension/count/size set matches a manual filesystem scan.
BFT-B100-019	Filtering	GUI checklist unchecks .json; Create Bundle run.	Resulting bundle excludes .json files; CLI run with equivalent --exclude-ext .json produces an identical manifest.

6. Repository (Git) Interaction — Assessment

6.1 Candidate scope

- Read-only, opt-in only. No write operations (no auto-commit, auto-stage, or auto-push) in any phase currently under consideration.
- Respect .gitignore as an additional, optional deny-pattern source when discovering files (opt-in flag/config key, not a default).
- “Changed-since” bundling: bundle only files changed relative to a given ref (working tree vs. HEAD, or vs. a named branch/tag) — high value for LLM-context and code-review workflows.
- Optional branch/commit metadata recorded in bundle headers for provenance, mirroring the project's existing provenance-documentation habits.

6.2 Why not Build 100

Paul's spec already lists explicit non-goals for Build 100 (Tree-sitter compression, MCP server mode, cloud processing, multi-user auth) specifically to protect the facade/profile/UI-parity work from scope creep. Repository interaction belongs in that same non-goals category for the same reason: it introduces a new external dependency (git CLI or GitPython), a new platform-difference surface (the team's own directive prioritizes Windows 11 / command.com first, cross-platform later, and git invocation differs meaningfully across those), and a new safety surface (reading arbitrary .gitignore content) — none of which should compete with Phase 1–4 priorities.

6.3 Recommendation

Capture as a named future feature, BFT-2.1-B101-REPO-AWARE, explicitly deferred until after Build 100's service facade and UI parity are complete and signed off. Add it to §5.4 of the Build 100 spec as an explicit non-goal (currently absent) so no team member assumes it is in scope. Revisit sizing and design once BundleToolService exists, since repo-awareness should also be implemented as facade methods (e.g., discover_files(request) gaining a changed_since_ref parameter) rather than a separate code path.

7. Recommendations Summary

- Approve Paul's Build 100 facade/CLI/Tkinter/Web direction as-is; no change recommended to that core architecture.
- Adopt this addendum's resolution order for file filtering (Section 5.2), pending George's sign-off, before Phase 2 work begins.
- Decide the default filtering posture (allow-list vs. deny-list, Section 5.8) explicitly — do not let it default silently through migration.
- Add BFT-2.1-B101-REPO-AWARE as a named, explicitly-deferred future feature; add it to the Build 100 non-goals list.
- Add the five gap items in Section 4 to Paul's next QA/QC pass before Phase 1 exit gate.
- On next scan, re-confirm bundle_frame.py's current contents before Phase 4 work, since it was unavailable for direct review in this round.

8. Change Log for This Deliverable

Date	Author	Change type	Description
2026-07-22	John (Lead Dev)	analysis/specification	Created Addendum B: gap review, repository-awareness assessment, and file inclusion/exclusion design, in response to Ringo's design-round request. No repository code files changed.